

WYKŁAD 2

Lokalizacja, odometria i fuzja sensoryczna

dr inż. Mateusz Pomianek



Estymacja stanu

Filtry EKF/UKF

Fuzja
sensoryczna

SLAM

Plan wykładu

Sekcja 1

Estymacja stanu: odometria humanoida, dryft, dead reckoning, układy transformacji

Sekcja 2

Filtry estymacyjne: komplementarny, EKF, UKF, cząsteczkowy, fuzja IMU+enkodery+wzrok

Sekcja 3

Lokalizacja globalna, SLAM, pose graph, domykanie pętli, dobór metody

Kody Python

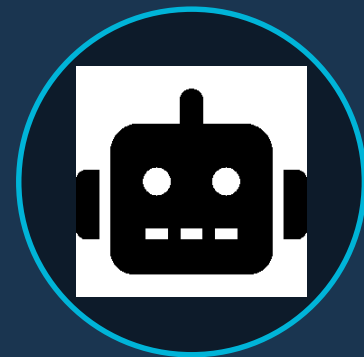
Całkowanie IMU, odometria kroku, filtr komplementarny, EKF 2D, korekta z landmarkem

01

SEKCJA

Estymacja stanu robota

Odometria, dryft, dead reckoning i układy transformacji



Czym jest stan robota humanoidalnego?

Pozycja i orientacja

x, y, z [m] oraz kąty roll, pitch, yaw [rad] w przestrzeni 3D

Prędkości

liniowa v [m/s] i kątowa ω [rad/s]:
wejście do modelu predykcji

Konfiguracja stawów

Kąty $q(t)$ [rad]: kinematyka prosta, Jacobian, zakres ruchu nóg

Stan kontaktu stóp

Lewa/prawa stopa: podparta czy w locie? Kluczowe dla odometrii

Środek masy (CoM)

Pozycja CoM w przestrzeni; związana z warunkiem stabilności ZMP

Niepewność estymacji

Macierz kowariancji P : miara zaufania do szacowanego stanu

Odometria humanoida vs. odometria kołowa

Odometria kołowa (AGV)

- Zliczanie impulsów enkoderów napędowych
- Proste równania kinematyczne (v , ω)
- Dryf głównie z poślizgu i nierówności kół
- Przewidywalny, mały błąd na krótkich dystansach
- Dojrzałe algorytmy korekcji (AMCL, EKF)

Odometria humanoida

- Integracja kroków + IMU + kontakty stóp
- Stan kontaktu: stopa oparta = referencyjna
- Dryf ze śliskich podłóg, wibracji tułowia
- Niesymetria kroków i ugięcie struktury
- Wymagana fuzja z wizją lub LiDAR-em

Dryft i błędy estymacji

- **Kumulacja błędów:** każde całkowanie dodaje szum; po 100 krokach błąd pozycji może wynosić $>0,5\text{m}$
- **Poślizg stopy:** nagła zmiana kontaktu stopy powoduje skok w estymowanej pozycji bazowej
- **Błędy kalibracji:** nieprecyzyjne IMU (niezrównoważenie osi) lub enkodery (luz w przekładni)
- **Szum żyroskopu:** dryft yaw przy całkowaniu prędkości kątowej – typowo $0,1\text{--}1^\circ/\text{s}$
- **Opóźnienia transmisji:** przesunięcie fazowe między sensorami zakłóca fuzję
- **Model kinematyczny:** uproszczenia (sztywne ogniwa, brak ugięcia) powodują systematyczny błąd
- **Zmiana środowiska:** ruchome podłoże (dywan, rampa) nie jest uwzględnione w modelu

Idea i ograniczenia Dead reckoning

Na czym polega

- Estymacja pozycji wyłącznie z modelu i pomiarów wewnętrznych (*IMU, enkodery*)
- Brak korekty z zewnętrznych punktów odniesienia
- Całkowanie prędkości: $x(t) = x_0 + \int v(\tau) d\tau$
- W krótkim horyzoncie czasowym: wystarczające
- Zastosowanie: awaryjne przejście przez strefę bez GPS/LiDAR

Dlaczego nie wystarczy

- Błąd rośnie z kwadratem czasu (*całkowanie szumu*)
- Bez korekty dryft yaw na poziomie kilku stopni/min
- Po 30 s marszu błąd pozycji może przekroczyć 1m
- Nie wykrywa zakłóceń środowiskowych (*poślizg*)
- Konieczna fuzja z SLAM, GPS lub znacznikami

Referencyjne układy: map, odom, base

- **world** – absolutna ramka inercjalna; stała; punkt odniesienia misji
- **map** – ramka mapy SLAM; skokowe korekty przy domknięciu pętli; world $\approx$ map w małych środowiskach
- **odom** – ramka odometrii; ciągła, ale dryfuje; baza dla planisty lokalnego
- **base** – miednica / CoM robota; centrum drzewa kinematyki
- **Relacja kluczowa:** odom $\rightarrow$ map to korekta SLAM; base $\rightarrow$ odom to bieżąca odometria
- **ROS 2 tf2:** każda transformacja niesie timestamp; stare transformacje są buforowane
- **Niespójne drzewo TF** jest najczęstszą przyczyną błędów nawigacji w ROS-ie

Kontakt stopy jako źródło informacji

- **Faza podparcia:** stopa oparta na podłożu – może służyć jako punkt odniesienia odometrii
- **Faza przenoszenia:** stopa w locie – brak danych pozycyjnych z tej nogi
- **Czujniki F/T w stopie mierzą 6-DoF:** F_x , F_y , F_z , M_x , M_y , M_z
- **Detekcja kontaktu:** $F_z > \text{próg}$ (np. 30 N) $\rightarrow$ stopa oparta; przydatne przy miękkich nawierzchniach
- **Kinematyczna odometria kroków:** pozycja bazy = pozycja stopy + FK^{-1}
- **Brak kontaktu obu stóp jednocześnie:** skok lub upadek – należy zresetować estymator
- **Fuzja z IMU:** stopa jako warunek brzegowy do korekcji dryftu yaw i translacji

Metryki jakości lokalizacji

RMSE pozycji

Pierwiastek średniokwadratowego błędu xy [m] względem ścieżki referencyjnej

Błąd orientacji

Średni błąd kąta yaw [°];
krytyczny dla planowania kroków

Dryft na minutę

Narastanie błędu pozycji bez korekty zewnętrznej [m/min]

Spójność estymatora

Czy kowariancja P odpowiada rzeczywistemu błędowi? (test NIS/NEES)

Całkowanie prędkości kątowej z IMU

```
import numpy as np

class ImuIntegrator:
    def __init__(self):
        self.yaw = 0.0 # [rad]
        self.pitch = 0.0 # [rad]
        self.roll = 0.0 # [rad]

    def update(self, gyro_xyz: np.ndarray, dt: float) -> tuple:
        """Aktualizacja orientacji przez całkowanie żyroskopu."""
        # gyro_xyz = [omega_x, omega_y, omega_z] w rad/s
        self.roll += gyro_xyz[0] * dt
        self.pitch += gyro_xyz[1] * dt
        self.yaw += gyro_xyz[2] * dt
        return self.roll, self.pitch, self.yaw

# Przykład użycia:
integrator = ImuIntegrator()
roll, pitch, yaw = integrator.update(np.array([0.01, -0.02, 0.15]), dt=0.005)
```

Opis: Proste całkowanie żyroskopu → szybkie, lecz podatne na dryft bez korekty zewnętrznej

Odometria kroku - aktualizacja pozycji robota

```
import numpy as np

class StepOdometry:
    def __init__(self):
        self.x = 0.0 # [m]
        self.y = 0.0 # [m]
        self.yaw = 0.0 # [rad]

    def step(self, stride_len: float, delta_yaw: float) -> tuple:
        """Aktualizacja pozycji po jednym kroku (stride)."""
        self.yaw += delta_yaw
        self.x += stride_len * np.cos(self.yaw)
        self.y += stride_len * np.sin(self.yaw)
        return self.x, self.y, self.yaw

odo = StepOdometry()
for _ in range(10):
    x, y, yaw = odo.step(stride_len=0.35, delta_yaw=0.02)
```

Opis: Każdy krok aktualizuje pozycję bazy przez kinematykę planarną; **delta_yaw** pochodzi z IMU

02

SEKCJA

Filtry estymacyjne i fuzja sensoryczna

*EKF, UKF, filtr cząsteczkowy, fuzja
IMU+enkodery+wzrok*



Po co łączyć sensory?

Komplementarność

Żyroskop: szybki, dryfuje. Akcelerometr: wolny, stabilny kąt. Razem: lepiej.

Redundancja

Awaria jednego sensora nie blokuje estymacji, jeśli mamy zapas informacji.

Wzajemna korekta

Kamera koryguje skumulowany dryft IMU; IMU stabilizuje estymację przy rozmazaniu

Pewność statystyczna

Kowariancja złożona < kowariancja pojedynczego sensora (*reguła filtracji*)

Filtr komplementarny - intuicja

- **Idea:** połącz żyroskop (szybki, dryfuje) z akcelerometrem (wolny, stabilny w długim czasie)
- **Wzór:** $\theta(t) = \alpha \cdot (\theta_{\text{prev}} + \omega_{\text{gyro}} \cdot dt) + (1-\alpha) \cdot \theta_{\text{accel}}$
- **Parametr α :** blisko 1 (np. 0,98) – większe zaufanie do żyroskopu na krótkich interwałach
- **Działanie:** żyroskop śledzi szybkie zmiany; akcelerometr koryguje powolny dryft
- **Zalety:** niska złożoność obliczeniowa, brak macierzy – działa na mikrokontrolerze IMU
- **Wada:** nie uwzględnia niepewności pomiarowych (brak kowariancji) ani dynamiki zewnętrznej
- **Zastosowanie:** pochylenie tułowia robota w czasie rzeczywistym (roll i pitch)

Filtr komplementarny dla kąta pochylenia

```
import numpy as np

class ComplementaryFilter:
    def __init__(self, alpha: float = 0.98):
        self.alpha = alpha
        self.pitch = 0.0 # estymowany kat pochylenia [rad]

    def update(self, gyro_y: float, accel_xz: np.ndarray, dt: float) -> float:
        """Estymacja pitch z gyro i akcelerometru."""
        # Predykcja z żyroskopu
        pitch_gyro = self.pitch + gyro_y * dt
        # Pomiar kąta z akcelerometru (g_x i g_z)
        pitch_accel = np.arctan2(accel_xz[0], accel_xz[1])
        # Fuzja
        self.pitch = self.alpha * pitch_gyro + (1 - self.alpha) * pitch_accel
        return self.pitch
```

Opis: $\alpha \approx 0,98$: 98% z żyroskopu (*szybkość*) i 2% z akcelerometru (*długoterminowa stabilność*)

Porównanie filtrów: EKF, UKF, cząsteczkowy

EKF - Extended Kalman Filter

- **Linearyzacja Jacobianem** wokół aktualnego stanu
- **Niski koszt $O(n^2)$** : działa w czasie rzeczywistym
- Zakłada **Gaussowski rozkład szumu**
- **Zawodzi** przy silnej **nieliniowości modelu**
- **Złoty standard w robotyce mobilnej**

UKF - Unscented Kalman Filter

- **Sigma-points** zamiast **linearyzacji**
- **Lepsza** aproksymacja **nieliniowości**
- **Bez** konieczności wyliczania **Jacobiana**
- **Większy koszt** niż EKF ($2n+1$ punktów)
- **Zalecany** przy mocno **nieliniowych dynamikach**

Filtr cząsteczkowy (PF)

- **Wielomodalny** rozkład prawdopodobieństwa
- **N cząsteczek** jako hipotezy stanu
- **Resampling**: eliminacja słabych hipotez
- **Koszt $O(N)$** : wymaga wielu cząsteczek
- Idealny do lokalizacji globalnej (AMCL)

Filtr Kalmana: cykl predykcja-korekcja

1

Predykcja

Model ruchu propaguje stan $\hat{x}$ i kowariancję P do przodu w czasie.

$$\hat{x}^- = F \cdot \hat{x} + B \cdot u$$

2

Pomiar

Sensor dostarcza nową obserwację z – zawierającą szum R .
Innowacja: $v = z - H \cdot \hat{x}^-$

3

Wzmocnienie K

Wagi Kalmana wyznaczają optymalny kompromis między modelem, a pomiarem.
 $K = P^- \cdot H^T \cdot (H \cdot P^- \cdot H^T + R)^{-1}$

4

Korekcja

Stan i kowariancja zaktualizowane o innowację.
 $\hat{x} = \hat{x}^- + K \cdot v$
 $P = (I - K \cdot H) \cdot P^-$

Uproszczony EKF dla pozycji 2D

```
import numpy as np

class EKF2D:
    def __init__(self):
        self.x = np.zeros(3)          # [x, y, yaw]
        self.P = np.eye(3) * 0.5      # kowariancja poczatkowa
        self.Q = np.diag([0.01, 0.01, 0.005]) # szum procesu
        self.R = np.diag([0.1, 0.1])  # szum pomiaru (x, y)

    def predict(self, v: float, omega: float, dt: float):
        yaw = self.x[2]
        self.x += np.array([v*np.cos(yaw)*dt, v*np.sin(yaw)*dt, omega*dt])
        F = np.eye(3); F[0,2]=-v*np.sin(yaw)*dt; F[1,2]=v*np.cos(yaw)*dt
        self.P = F @ self.P @ F.T + self.Q

    def correct(self, z_xy: np.ndarray):
        H = np.array([[1,0,0],[0,1,0]])
        S = H @ self.P @ H.T + self.R
        K = self.P @ H.T @ np.linalg.inv(S)
        self.x += K @ (z_xy - self.x[:2])
        self.P = (np.eye(3) - K @ H) @ self.P
```

Opis: predict() używa modelu jednośladowego (unicycle); correct() koryguje pomiar GPS lub znacznika

Fuzja IMU, enkoderów i wizji

- **IMU (200 Hz):** predykcja stanu – szybka propagacja orientacji i przyspieszenia
- **Enkodery + kontakty stóp (100–500 Hz):** kinematyczna odometria kroków – korekta translacji
- **Kamera (30 Hz) / LiDAR (10 Hz):** obserwacje pozycyjne – usuwanie skumulowanego dryfu
- **Architektura loosely coupled:** każde źródło dostarcza niezależnej estymatę pozycji do EKF
- **Architektura tightly coupled:** surowe pomiary sensorów trafiają bezpośrednio do jednego filtra
- **Opóźnienia:** pomiar z kamery z 33 ms opóźnieniem wymaga cofnięcia stanu (state rewinding)
- **Stan złożony:** $x = [\text{pos}, \text{vel}, \text{orient}, \text{biasy IMU}]$ – typowo 15–21 zmiennych stanu

Wykrywanie awarii sensora

Test innowacji (chi-squared)

- Innowacja $v = z - H \cdot \hat{x}^-$ powinna być zbliżona do zera
- Statystyka χ^2 : $v^{-1} \cdot S^{-1} \cdot v > \text{próg} \rightarrow \text{sensor awaryjny}$
- Zaimplementowane w bibliotekach iSAM2, GTSAM
- Działa online, bez zatrzymywania robota

Tryb degradacji (graceful degradation)

- Awaria kamery $\rightarrow$ nawigacja tylko na IMU + enkodery
- Awaria LiDAR $\rightarrow$ zmniejszony zasięg planowania
- Awaria jednej stopy F/T $\rightarrow$ wyłącz kinematyczną odo dla tej nogi
- Zwiększenie macierzy Q i P $\rightarrow$ większy obszar niepewności

03

SEKCJA

Lokalizacja globalna i SLAM

ICP, visual odometry, pose graph, domykanie pętli



Lokalizacja względem znanych map

1. **Cel:** wyznaczenie pozycji robota w już zbudowanej mapie środowiska
2. **Monte Carlo Localization (*MCL / AMCL*):** filtr cząsteczkowy z obserwacjami LiDAR
3. **Matching 2D:** porównanie aktualnego skanu z mapą zajętości – scoring hipotez
4. **Matching 3D (*ICP*):** dopasowanie chmury punktów do globalnej mapy PCL
5. **Histogram Filter (*HMM*):** dyskretne komórki mapy jako hipotezy lokalizacji
6. **Warunek dobrej lokalizacji:** mapa aktualna i środowisko o charakterystycznych elementach
7. **Kidnapped robot problem:** wymagana inicjalizacja globalna, gdy pozycja startowa jest nieznana

ICP – Iterative Closest Point

- **Cel:** znalezienie transformacji (R, t) minimalizującej odległości między dwoma chmurami punktów
- **Krok 1 – Korespondencja:** dla każdego punktu źródłowego znajdź najbliższy punkt w chmurze docelowej
- **Krok 2 – Estymacja:** oblicz optymalną transformację (metoda SVD lub minimalizacja MLS)
- **Krok 3 – Aktualizacja:** zastosuj transformację i oblicz błąd residualny
- **Iteruj do zbieżności** (błąd $<$ próg) lub **osiągnięcia max. iteracji**
- **Wady:** wolna zbieżność, wrażliwość na punkt startowy (koniecznie dobre wstępne wyrównanie)
- **Warianty:** Point-to-Plane ICP, GICP – znacznie szybsze i bardziej niezawodne

Odometria wizyjna (Visual Odometry)

Monokularna VO

- Śledzenie cech ORB/SIFT między klatkami
- Essential Matrix $\rightarrow$ ruch kamery R, t
- Skala ruchu nieznana (do 5-punktowego)
- Skumulowany dryft bez domknięcia pętli
- Niska cena – jedna kamera wystarczy

Stereofoniczna / RGB-D VO

- Bezpośrednia metryczna skala ruchu
- Głębina z paralaksy lub strukturalizowanego światła
- Mniejszy dryft niż mono VO
- Wymagana synchronizacja i kalibracja stereo
- Podstawa Visual-Inertial Odometry (VIO)

SLAM – Simultaneous Localization and Mapping

- **Problem sprzężony:** mapa zależy od dobrej lokalizacji; lokalizacja zależy od dobrej mapy
- **Online SLAM:** estymacja bieżącej pozycji i mapy w czasie rzeczywistym
- **Full SLAM:** pełna trajektoria i mapa (post-processing w np. Google Cartographer)
- **Dane obserwacyjne:** LiDAR (2D/3D SLAM), kamera (Visual SLAM – ORB-SLAM3, RTAB-Map)
- **Reprezentacja mapy:** chmura punktów, mapa cech (deskryptory), submapa siatek
- **Współdziałanie z planistą:** gotowa mapa przekazywana do globalnego planisty ścieżki
- **Błąd rośnie liniowo z odległością** bez domknięcia pętli; po 100 m może sięgnąć kilku metrów

Pose Graph SLAM - idea grafowa

1

Węzły (*poses*)

Każda pozycja robota $x_1, x_2, \dots, x_n$ to wierzchołek w grafie; zazwyczaj SE(2) lub SE(3)

2

Krawędzie (*constraints*)

Relacja między dwoma pozycjami: odometria, ICP, wizja. Niesie kowariancję błędu.

3

Optymalizacja

Minimalizacja sumy błędów kwadratowych (least squares): g2o, GTSAM, iSAM2

4

Domknięcie pętli

Detekcja powrotu do miejsca; dodanie krawędzi pogrubionej; jednorazowa korekta dryfu

Domykanie pętli (*loop closure*)

Detekcja powrotu do miejsca

- **Bag-of-Words (DBoW2):** histogram deskryptorów wizualnych
- **Scan matching:** ICP między bieżącym, a historycznym skanem
- **Mapa cech:** dopasowanie deskryptorów ORB/SIFT
- **Próg podobieństwa:** zapobiega fałszywym detekcjom
- **Weryfikacja geometryczna:** RANSAC na korespondencjach

Korekta dryfu po domknięciu

- **Nowa krawędź** w pose graph z małą kowariancją
- **Optymalizator (g2o)** przelicza cały graf
- **Błąd dryfu** rozłożony wzdłuż zamkniętej pętli
- **Efekt:** mapa globalna spójna bez artefaktów
- **Częstość:** co kilka sekund dla środowisk cyklicznych

Prosta korekta pozycji po detekcji znacznika

```
import numpy as np

def correct_with_landmark(state: np.ndarray,
                          P: np.ndarray,
                          landmark_pos: np.ndarray,
                          measured_pos: np.ndarray,
                          R_meas: np.ndarray) -> tuple:
    """Korekta EKF po pomiarze pozycji znajomego znacznika."""
    H = np.eye(2, 3) # obserwujemy tylko x, y
    predicted_z = landmark_pos[:2]
    innovation = measured_pos - predicted_z
    S = H @ P @ H.T + R_meas
    K = P @ H.T @ np.linalg.inv(S) # wzmocnienie Kalmana
    state_new = state + K @ innovation
    P_new = (np.eye(3) - K @ H) @ P
    return state_new, P_new
```

Opis: Korekta EKF: innowacja z detekcji ArUco/AprilTag eliminuje skumulowany dryft odometrii

Dobór metody lokalizacji do zadania

Laboratorium

Znana, statyczna mapa; AMCL
z LiDAR-em 2D wystarczy;
opcjonalnie znaczniki ArUco

Biuro

Zmienna mapa (meble);
SLAM online (Cartographer)
+ MCL; UWB jako wsparcie

Magazyn

Pojazdy AGV i regały;
Visual SLAM z kamerami
panoramicznymi; QR na półkach

Schody

3D LiDAR SLAM (LeGO-LOAM)
lub RGB-D SLAM (RTAB-Map);
height map aktywna

Dynamiczne otoczenie

EKF + filtr cząsteczkowy;
odrzuć ruchomych obiektów
ze skanów mapowania

Zewnętrzne warunki

GPS + IMU (EKF) jako baza;
LiDAR do refinement;
odporność na warunki pogodowe

PODSUMOWANIE

- 01 Dryft odometrii jest nieunikniony – konieczna jest korekta z zewnętrznych sensorów lub SLAM
- 02 Filtr Kalmana (EKF/UKF) to złoty standard fuzji IMU, enkoderów i danych wizyjnych
- 03 Filtr cząsteczkowy radzi sobie z wielomodalnością, lecz wymaga większych zasobów obliczeniowych
- 04 SLAM (jednoczesne mapowanie i lokalizacja) – pozwala robotowi budować i korygować mapę w locie
- 05 Domykanie pętli eliminuje skumulowany dryft; awarię sensora wykrywa się przez test innowacji

Pytania i mini-studium przypadku

- Dlaczego filtr komplementarny nie może zastąpić EKF w pełnym systemie lokalizacji?
- Jak wzmocnienie Kalmana K zmienia się, gdy macierz szumu pomiaru R jest bardzo duża?
- Co to jest NIS (Normalized Innovation Squared) i jak wykrywa awarię sensora?
- Opisz, jakie sensory i jakie filtry wybrać, dla humanoida patrolującego budynek przez 30 min, aby błąd pozycji nie przekroczył 0,3m. Jakie jest minimalne wymaganie na częstotliwość domknięcia pętli?

Zadanie

Zaimplementuj StepOdometry z Sekcji 1 i porównaj dryft z i bez korekty z EKF przy symulowanym szumie $\text{yaw} = 0,02 \text{ rad/krok}$.